UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

Programación I

Proyecto Semestral – Avance #1

Sistema de Gestión de Restaurantes

Integrantes: Bryan Mark · Gabriel Ibarra

Fecha: 8 de octubre de 2026

---

1. Descripción del sistema

El Sistema de Gestión de Restaurantes controla las mesas, los pedidos, el menú, la cocina, la facturación y los empleados de un restaurante. Permite registrar los pedidos de cada mesa, llevar el inventario de ingredientes, facturar y cerrar cada turno, y obtener reportes de los platos más vendidos.

El modelo tiene 14 clases: las 12 sugeridas en el escenario (Mesa, Pedido, Plato, Categoria, Cocinero, Mesero, Factura, Cliente, Inventario, Ingrediente, Turno y Administrador) y dos que agregamos. Empleado es una clase abstracta de la que heredan Administrador, Mesero y Cocinero, para no repetir sus datos comunes; DetallePedido guarda cada plato de un pedido con su cantidad.

2. Tabla 1. Escenarios, clases involucradas y explicación

Escenario	Clases involucradas	Explicación
Registro de pedidos por mesa	Mesa, Pedido, DetallePedido, Plato, Mesero, Cliente	El Mesero toma el pedido en una mesa libre con tomarPedido(), que crea el Pedido asociado a la Mesa y al Cliente y deja la mesa ocupada. Cada plato solicitado se agrega con agregarPlato() como un DetallePedido (plato, cantidad y observación). El pedido calcula su subtotal y cambia de estado hasta que el mesero lo entrega.
Control de inventario de ingredientes	Inventario, Ingrediente, Plato, Pedido, Cocinero	El Inventario guarda cada Ingrediente con su cantidad disponible y su stock mínimo. Cuando el Cocinero prepara un Pedido, descontarIngredientes() rebaja lo que usa cada Plato; si algún ingrediente no alcanza, el pedido no pasa a preparación. listarIngredientesPorAgotarse() indica cuáles necesitan reposición y registrarEntrada() suma las compras.
Facturación y cierre de turno	Factura, Pedido, Cliente, Mesero, Mesa, Turno	El Mesero cobra con cobrarPedido(), que genera la Factura: toma el subtotal del Pedido, calcula el impuesto (ITBMS) y el total, registra el método de pago con los datos del Cliente y libera la Mesa. Cada factura se registra en el Turno abierto; cerrarTurno() suma las ventas del turno y lo deja cerrado.

Escenario	Clases involucradas	Explicación
Reportes de platos más vendidos	Plato, DetallePedido, Factura, Turno, Categoria, Administrador	Al facturar, registrarVenta() suma en cada Plato las unidades indicadas en su DetallePedido. El Turno recorre sus facturas y obtenerPlatosMasVendidos() devuelve el ranking; el Administrador lo solicita con generarReportePlatosMasVendidos(). obtenerPlatoMasVendido() de Categoria indica el plato más vendido de cada sección del menú.
Asignación de roles y control de desempeño	Empleado, Administrador, Mesero, Cocinero, Turno	Empleado es la clase abstracta de la que heredan Administrador, Mesero y Cocinero: el rol de cada persona lo define su subclase y obtenerRol() lo informa. El Administrador asigna a cada empleado a un Turno con asignarTurno(). El desempeño se mide con calcularDesempeno(), que cada rol implementa a su manera: el mesero según los pedidos atendidos y el cocinero según los platos preparados.
Gestión de menú y personal	Administrador, Categoria, Plato, Empleado	El Administrador mantiene el menú: organiza las Categorias, agrega cada Plato a su categoría con agregarPlatoAlMenu() y cambia precios con actualizarPrecio(). También gestiona el personal: registra empleados con registrarEmpleado(), los da de baja con darDeBajaEmpleado() y cada Empleado entra al sistema con iniciarSesion().

3. Tabla 2. Clases, atributos y métodos

Modificadores de acceso: + public (público) · - private (privado) · # protected (protegido).

Además de los métodos listados, cada clase tiene su constructor y los getters y setters públicos de sus atributos.

Clase	Atributos	Métodos
Empleado *Clase abstracta*	# id: int # nombre: String # cedula: String # usuario: String - clave: String # activo: boolean	+ iniciarSesion(usuario: String, clave: String): boolean + cambiarClave(claveActual: String, claveNueva: String): boolean + obtenerRol(): String (abstracto) + calcularDesempeno(): double (abstracto)
Administrador *Hereda de Empleado*	- nivelAcceso: int - personal: List<Empleado> - menu: List<Categoria>	+ obtenerRol(): String + calcularDesempeno(): double + registrarEmpleado(empleado: Empleado): void + darDeBajaEmpleado(empleado: Empleado): void + asignarTurno(empleado: Empleado, turno: Turno): void + agregarPlatoAlMenu(plato: Plato, categoria: Categoria): void + consultarDesempeno(empleado: Empleado): double + generarReportePlatosMasVendidos(turno: Turno): List<Plato>

Clase	Atributos	Métodos
Mesero *Hereda de Empleado*	- mesasAsignadas: List<Mesa> - pedidosAtendidos: int	+ obtenerRol(): String + calcularDesempeno(): double + asignarMesa(mesa: Mesa): void + tomarPedido(mesa: Mesa, cliente: Cliente): Pedido + entregarPedido(pedido: Pedido): void + cobrarPedido(pedido: Pedido, metodoPago: String): Factura
Cocinero *Hereda de Empleado*	- especialidad: String - platosPreparados: int	+ obtenerRol(): String + calcularDesempeno(): double + prepararPedido(pedido: Pedido, inventario: Inventario): boolean + marcarPedidoListo(pedido: Pedido): void
Cliente	- id: int - nombre: String - cedula: String - telefono: String	+ actualizarDatos(nombre: String, telefono: String): void + obtenerDatosFactura(): String
Mesa	- numero: int - capacidad: int - estado: String	+ ocupar(): void + liberar(): void + estaDisponible(): boolean
Pedido	- numero: int - fechaHora: LocalDateTime - estado: String - mesa: Mesa - mesero: Mesero - cliente: Cliente - detalles: List<DetallePedido>	+ agregarPlato(plato: Plato, cantidad: int, observacion: String): void + quitarPlato(plato: Plato): void + calcularSubtotal(): double + cambiarEstado(nuevoEstado: String): void
DetallePedido	- plato: Plato - cantidad: int - observacion: String	+ calcularImporte(): double
Plato	- codigo: int - nombre: String - precio: double - disponible: boolean - vecesVendido: int - ingredientes: List<Ingrediente>	+ agregarIngrediente(ingrediente: Ingrediente): void + actualizarPrecio(nuevoPrecio: double): void + registrarVenta(cantidad: int): void
Categoria	- id: int - nombre: String - descripcion: String - platos: List<Plato>	+ agregarPlato(plato: Plato): void + quitarPlato(plato: Plato): void + listarPlatosDisponibles(): List<Plato> + obtenerPlatoMasVendido(): Plato

Clase	Atributos	Métodos
Ingrediente	- codigo: int - nombre: String - unidadMedida: String - cantidadDisponible: double - stockMinimo: double	+ aumentarStock(cantidad: double): void + descontarStock(cantidad: double): boolean + necesitaReposicion(): boolean
Inventario	- ingredientes: List<Ingrediente> - fechaActualizacion: LocalDate	+ registrarIngrediente(ingrediente: Ingrediente): void + registrarEntrada(ingrediente: Ingrediente, cantidad: double): void + descontarIngredientes(plato: Plato, cantidad: int): boolean + listarIngredientesPorAgotarse(): List<Ingrediente>
Factura	- numero: int - fecha: LocalDateTime - pedido: Pedido - subtotal: double - impuesto: double - total: double - metodoPago: String - pagada: boolean	+ calcularImpuesto(): double + calcularTotal(): double + registrarPago(metodoPago: String): void + generarComprobante(): String
Turno	- id: int - fecha: LocalDate - horaInicio: LocalTime - horaFin: LocalTime - abierto: boolean - empleados: List<Empleado> - facturas: List<Factura>	+ asignarEmpleado(empleado: Empleado): void + registrarFactura(factura: Factura): void + calcularTotalVentas(): double + obtenerPlatosMasVendidos(cantidad: int): List<Plato> + cerrarTurno(): double

4. Diagrama de clases UML

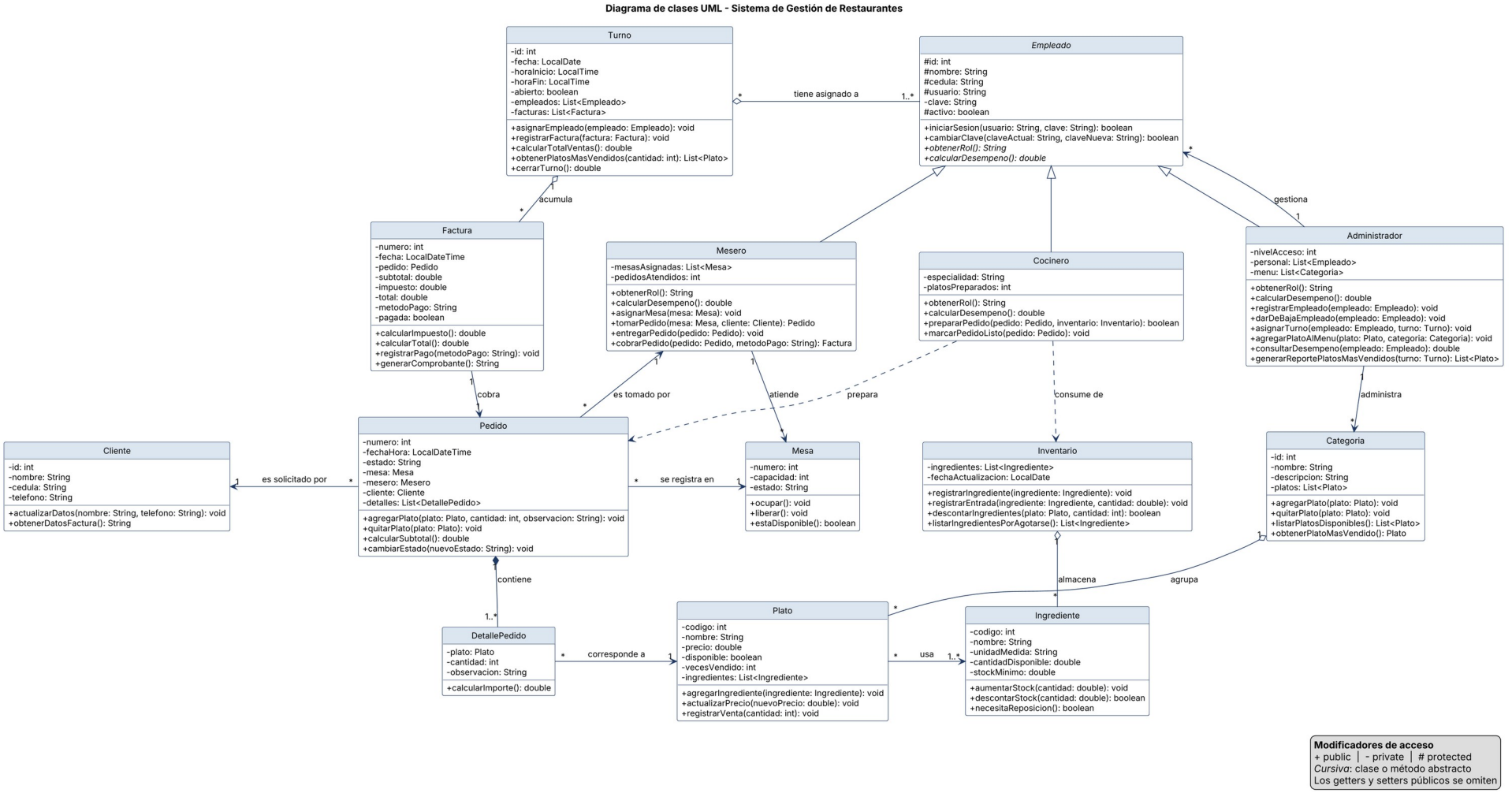


Figura 1. Diagrama de clases del sistema (docs/uml/diagrama-clases.png).

5. Relaciones entre clases

Las asociaciones del diagrama, con su tipo y multiplicidad (origen : destino).

Relación	Tipo	Multiplicidad	Significado
Administrador → Empleado	Herencia	—	Administrador es un tipo de Empleado
Mesero → Empleado	Herencia	—	Mesero es un tipo de Empleado
Cocinero → Empleado	Herencia	—	Cocinero es un tipo de Empleado
Administrador → Empleado	Asociación	1 : *	Administrador gestiona Empleado
Administrador → Categoria	Asociación	1 : *	Administrador administra Categoria
Mesero → Mesa	Asociación	1 : *	Mesero atiende Mesa
Pedido → Mesa	Asociación	* : 1	Pedido se registra en Mesa
Pedido → Mesero	Asociación	* : 1	Pedido es tomado por Mesero
Pedido → Cliente	Asociación	* : 1	Pedido es solicitado por Cliente
Pedido → DetallePedido	Composición	1 : 1..*	Pedido contiene DetallePedido
DetallePedido → Plato	Asociación	* : 1	DetallePedido corresponde a Plato
Categoria → Plato	Agregación	1 : *	Categoria agrupa Plato
Plato → Ingrediente	Asociación	* : 1..*	Plato usa Ingrediente
Inventario → Ingrediente	Agregación	1 : *	Inventario almacena Ingrediente
Factura → Pedido	Asociación	1 : 1	Factura cobra Pedido
Turno → Empleado	Agregación	* : 1..*	Turno tiene asignado a Empleado
Turno → Factura	Agregación	1 : *	Turno acumula Factura
Cocinero → Pedido	Dependencia	—	Cocinero prepara Pedido
Cocinero → Inventario	Dependencia	—	Cocinero consume de Inventario

6. Estructura de carpetas del repositorio

El repositorio de GitHub sigue la estructura del ejemplo de la asignatura: el código en src, y el diagrama UML y este documento en docs.

```
Gestion-restaurantes/
├── src/
│   ├── main/
│   ├── resources/
│   └── utp/
│       └── Ac/
│           └── Pa/
│               └── sistema/
│                   ├── App.java  (principal)
│                   ├── utils/
│                   │   └── IOUtils.java
│                   └── domain/
│                       ├── Empleado.java
│                       ├── Administrador.java
│                       ├── Mesero.java
│                       ├── Cocinero.java
│                       ├── Cliente.java
│                       ├── Mesa.java
│                       ├── Pedido.java
│                       ├── DetallePedido.java
│                       ├── Plato.java
│                       ├── Categoria.java
│                       ├── Ingrediente.java
│                       ├── Inventario.java
│                       ├── Factura.java
│                       └── Turno.java
├── docs/
│   ├── Avance1-Gestion-restaurantes.docx
│   ├── Avance1-Gestion-restaurantes.pdf
│   └── uml/
│       ├── diagrama-clases.png
│       └── diagrama-clases.puml
└── README.md
```

App es la clase principal y IOUtils reúne la entrada y salida por consola; las 14 clases del modelo están en el paquete domain.